Photogallery, un’applicazione a microservizi

Progetto per il corso di Sistemi Distribuiti e Cloud Computing

---

Matteo Orlando 0382170

Università degli studi di Roma Tor Vergata

October 14, 2026

Indice

1. Introduzione

2. Implementazione

3. Risultati

4. Conclusioni

Introduzione

Obiettivo

Lo scopo di questo progetto è implementare un sistema di notifiche per gallerie fotografiche realizzato come applicazione a microservizi in Go.

Requisiti non funzionali

- Implementazione in Go
- Comunicazione tra i servizi tramite gRPC
- Il sistema deve essere scalabile e adeguatamente testato
- Deploy tramite Docker su EC2

Requisiti funzionali

- Registrazione di utenti e moderatori
- Gestione delle gallerie da parte dei moderatori
- Caricamento di foto nelle gallerie
- Notifica ai membri della galleria dei nuovi caricamenti

Tecnologie adottate





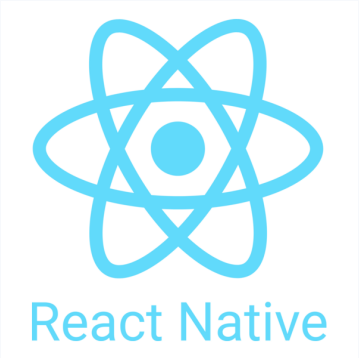








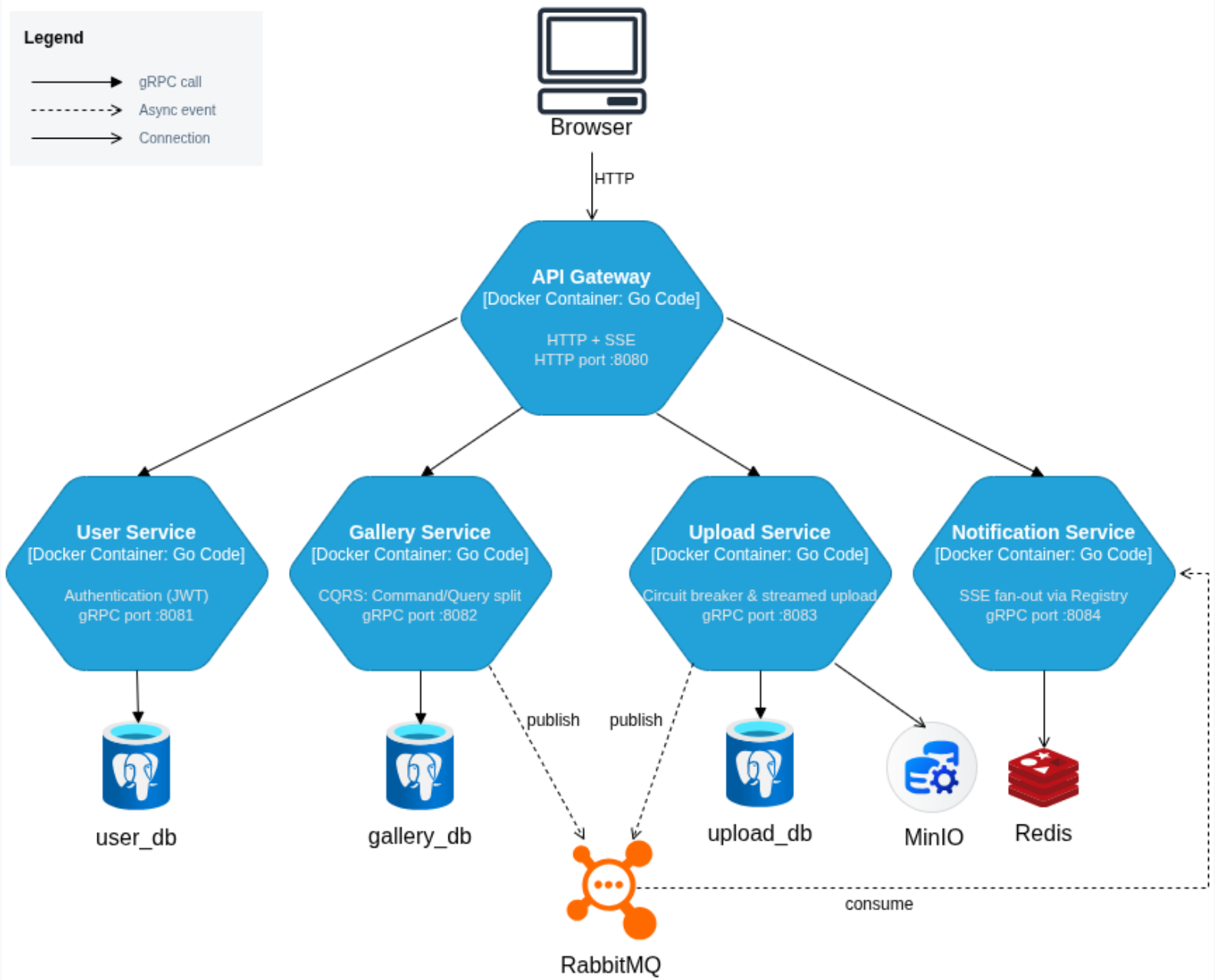




Pattern adottati

Pattern	Dove
CQRS	Gallery Service: separa scrittura e lettura
Circuit Breaker	Upload $\rightarrow$ Gallery: resilienza a fallimenti
Database per Service	Ogni microservizio che necessita persistenza
Flux	Frontend: flusso unidirezionale per gestire lo stato

Architettura



Implementazione

Framework e strumenti utilizzati

Framework	Ruolo
Buf	Generazione codice RPC
GORM	ORM per accesso a PostgreSQL
GoMock	Mock generati per unit test
grpc-gateway	Reverse proxy HTTP $\rightarrow$ gRPC
Gin	Route non-unary: upload, SSE

CQRS nel Gallery Service

Comandi (write)

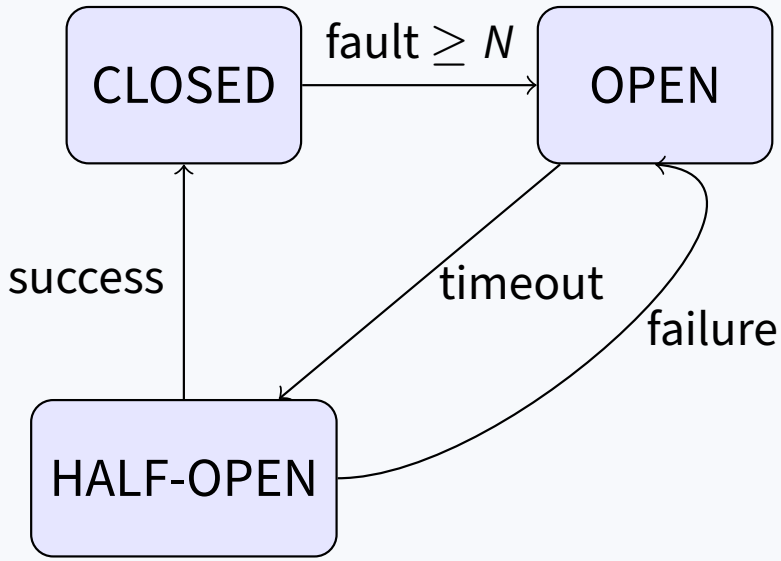
- CommandRepository
- Logica di business in CommandService
- Verifica autorizzazione prima di modificare lo stato
- Pubblicazione asincrona su RabbitMQ dopo il commit

Query (read)

- QueryRepository
- Logica di business in QueryService
- Operazioni idempotenti
- Recupero dati ottimizzato per la lettura

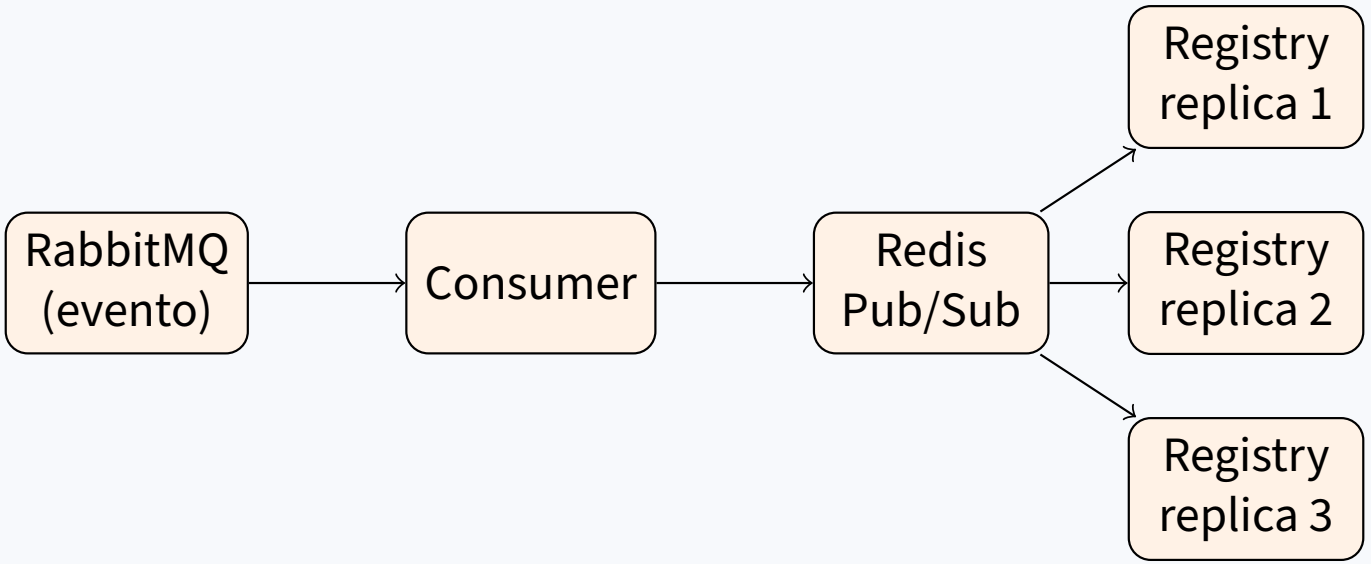
Circuit breaker

- Tre stati: **CLOSED**, **OPEN**, **HALF-OPEN**
- Mutex garantisce una sola probe alla volta in HALF-OPEN
- Gestisce i fallimenti senza bloccare le richieste sincrone



Distribuzione notifiche

- L'evento viene pubblicato su RabbitMQ in una goroutine in background, dopo il commit della transazione
- Il Notification Service consuma l'evento in modo asincrono e lo trasforma in una notifica per gli utenti interessati
- Redis Pub/Sub propaga la notifica a tutte le repliche del Notification Service
- Ogni replica attraverso il registro in memoria inoltra la notifica ai propri client



Metodologia di testing

- **Unit test**: dipendenze esterne mockate con GoMock
- **Test di integrazione**: client black-box contro lo stack HTTP reale
- **Load test**: generano carico concorrente pesato verso il Gateway, misurando throughput, latenza e tasso di errore.

Risultati

Load test: 1 replica vs 3 repliche

Servizio	Repliche	Throughput	Latenza (min/media/max)	Errori
Gallery	1	44.0 req/s	106.30 / 137.04 / 1144.52 ms	0%
Gallery	3	69.3 req/s	107.42 / 140.54 / 825.89 ms	0%
Upload	1	107.0 req/s	106.03 / 184.30 / 1229.10 ms	63.8%
Upload	3	111.7 req/s	106.00 / 182.45 / 1232.62 ms	47.9%

Limiti

- **Pubblicazione eventi non atomica**: un crash intermedio può far perdere un evento
- **Registry in memoria**: lo stato delle connessioni SSE va ricostruito ad ogni riconnessione
- **Single point of failure**: RabbitMQ, Redis e le tre istanze PostgreSQL non sono replicate

Compromessi

- **Ridondanza RabbitMQ + Redis**: ogni replica del Notification Service deve riapplicare localmente il filtraggio degli eventi ricevuti via Pub/Sub
- **Deployment single-host**: Docker Compose su una singola istanza EC2 semplifica la riproducibilità, ma non consente una vera distribuzione multi-host né una disponibilità da ambiente di produzione
- **Robustezza dei load test**: i risultati dipendono da dimensioni dell'istanza, numero di repliche e singola esecuzione

Conclusioni

Conclusioni

- CQRS permette la separazione tra logica di scrittura e di lettura, testing isolato dei due percorsi
- Circuit Breaker garantisce resilienza verso i malfunzionamenti del Gallery Service
- RabbitMQ e Redis Pub/Sub permettono la corretta distribuzione delle notifiche
- Suite di test permette di verificare la correttezza funzionale e il comportamento sotto scalabilità/guasto
- Sviluppi futuri: atomicità tra scrittura e pubblicazione eventi, separazione fisica del modello di lettura, migrazione a Kubernetes, monitoring/logging distribuito

Grazie per l’attenzione